Stack

leetcode

20.有效的括号

- 给定一个只包括 `'('`，`')'`，`'{'`，`'}'`，`'['`，`']'` 的字符串 `s` ，判断字符串是否有效。

```
输入: s = "()[]{}"

输出: true
```

- 思路：该算法利用 **栈 (Stack)** 的**“后进先出” (LIFO)** 特性来匹配括号。其基本逻辑是：**遇到左括号就压入栈中，遇到右括号就检查栈顶元素是否为对应的左括号。**

```
class Solution {
public:
    bool isValid(string s) {
        int n = s.size();
        if( n%2==1 ){ // 长度为偶数才能正确匹配
            return false;
        }

        unordered_map<char,char> rightToLeft = {
            {')', '('},
            {']', '['},
            {'}', '{'}
        };
        stack<char> stk;
        for( char ch: s){
            if(rightToLeft.count(ch)){ // 右括号则需要和栈中的左括号进行匹配
                if( stk.empty() || stk.top() != rightToLeft[ch] ){ // 栈为空/
栈顶左括号不匹配右括号
                    return false; // 匹配失败
                }
                stk.pop(); // 弹出栈顶元素
            }else{ // 把左括号压入栈中
                stk.push(ch);
            }
        }
        return stk.empty();
    }
};
```

-

155.最小栈

- 设计一个支持 `push` ，`pop` ，`top` 操作，并能在常数时间内检索到最小元素的栈。

  实现 `MinStack` 类：

  - `MinStack()` 初始化堆栈对象。
  - `void push(int val)` 将元素val推入堆栈。
  - `void pop()` 删除堆栈顶部的元素。
  - `int top()` 获取堆栈顶部的元素。
  - `int getMin()` 获取堆栈中的最小元素。

  输入：
  ```
  ["MinStack","push","push","push","getMin","pop","top","getMin"]
  [[],[-2],[0],[-3],[],[],[],[]]
  ```

  输出：
  ```
  [null,null,null,null,-3,null,0,-2]
  ```

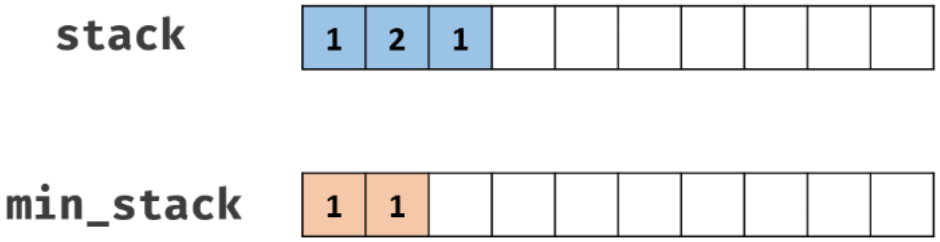
  解释：
  ```
  MinStack minStack = new MinStack();
  minStack.push(-2);
  minStack.push(0);
  minStack.push(-3);
  minStack.getMin();   --> 返回 -3.
  minStack.pop();
  minStack.top();      --> 返回 0.
  minStack.getMin();   --> 返回 -2.
  ```

- 思路：**辅助栈 (Auxiliary Stack)**.

  为了能在 $O(1)$ 时间内得到最小值，代码使用了**两个栈**同步工作：

  1. `valStk` (数据栈)：正常存储所有入栈的元素，维持栈的基本逻辑。
  2. `minStk` (辅助栈)：与 `valStk` 一一对应。 `minStk` 的第 i 个元素，存储的是 当 `valStk` 压入第 i 个元素后，整个栈内的最小值。

  - 当一个元素要入栈时，我们取当前辅助栈的栈顶存储的最小值，与当前元素比较得出最小值，将这个最小值插入辅助栈中；
  - 当一个元素要出栈时，我们把辅助栈的栈顶元素也一并弹出；
  - 在任意一个时刻，栈内元素的最小值就存储在辅助栈的栈顶元素中。



执行 stack.push(1)

```
class MinStack {
    stack<int> valStk; // 存储入栈的元素
    stack<int> minStk; // 辅助栈，用于存储当前元素入栈时对应的栈内的最小元素
public:
    MinStack() {
        minStk.push(INT_MAX);
    }

    void push(int val) {
        valStk.push(val);
        minStk.push(min(minStk.top(),val));
    }

    void pop() {
        valStk.pop();
        minStk.pop();
    }

    int top() {
        return valStk.top();
    }

    int getMin() {
        return minStk.top();
    }
};

/**
 * Your MinStack object will be instantiated and called as such:
 * MinStack* obj = new MinStack();
 * obj->push(val);
 * obj->pop();
 * int param_3 = obj->top();
 * int param_4 = obj->getMin();
 */
```

-

394.字符串解码

- 给定一个经过编码的字符串，返回它解码后的字符串。编码规则为: `k[encoded_string]`，表示其中方括号内部的 `encoded_string` 正好重复 `k` 次。

```
输入: s = "3[a2[c]]"
输出: "accaccacc"
```

- 思路:

1. 核心策略：遇 `]` 则展开

算法不预先解析整个字符串的结构，而是**线性遍历**字符串。

- 普通字符和 `[` ：直接压入栈中，暂时保存。
- 右括号 `]` ：这是一个触发信号，表示当前最内层的编码结构已经结束，需要立即进行“解码”操作。

第一步：遍历与压栈

遍历输入字符串 `s` 的每一个字符 `c` ：

- 如果 `c` 不是 `]` ，直接 `stk.push(c)` 。
  - 此时栈中可能混杂着字母、数字和左括号 `[` ，例如栈底到栈顶可能是: `['3', '[', 'a', '2', '[', 'c']` 。

第二步：遇到 `]` 时的解码逻辑（关键）

当遇到 `]` 时，说明最近的一个 `[` 及其内部的字符串、以及 `[` 前面的数字已经全部入栈。此时执行以下三个子步骤:

1. 提取待重复的字符串 (`sub`):
   - 不断弹出栈顶元素，直到遇到 `[` 或栈空。
   - 因为栈是“后进先出”，弹出的字符顺序是逆序的（例如栈里是 `... '[', 'c', 'b', 'a'` ，弹出得到 `"abc"` 的反序 `"cba"` ）。
   - 将弹出的字符拼接成字符串后，执行 `reverse` 还原为正确顺序（即 `sub = "abc"` ）。
   - 弹出 `[` ：此时栈顶必然是 `[` ，将其弹出丢弃，完成括号的匹配清除。
2. 提取重复倍数 (`cnt`):
   - 继续检查栈顶，如果是数字字符，则持续弹出。
   - 同样，弹出的数字字符也是逆序的（例如数字 `12` 在栈中是 `'1', '2'` ，弹出得到 `"21"` ）。
   - 拼接后执行 `reverse` 还原，并使用 `stoi` 转换为整数 `cnt` 。
   - *注意：这里假设数字紧邻 `[` 之前，符合题目格式规范。*
3. 重复并压回栈中:
   - 根据提取出的倍数 `cnt` ，将字符串 `sub` 重复 `cnt` 次。
   - 将重复后的每一个字符依次 `push` 回栈中。
   - 效果：原本栈中的 `数字 + [ + 字符串 + ]` 这一小段结构，被替换成了展开后的纯字符串。这使得外层的编码可以继续处理这个展开后的结果（支持嵌套）。

第三步：组装最终结果

- 遍历结束后，栈中剩下的就是完全解码后的字符序列，但顺序依然是逆序的（因为最后入栈的在栈顶）。
- 依次弹出栈中所有字符拼接到结果字符串 `res` 中。
- 对 `res` 执行最后一次 `reverse` ，得到最终的正序解码字符串。

```
class Solution {
public:
    string decodeString(string s) {
        stack<char> stk;
```

```
        for(char c:s){
            // 把所有字符push进去，除了']'
            if (c != ']'){
                stk.push(c);
            }else{
                // 取出[]内的字符串
                string sub="";
                while(!stk.empty() && isalpha(stk.top())){
                    sub += stk.top();
                    stk.pop();
                }
                // 反转字符串（从栈顶弹出，先进后出）
                reverse(sub.begin(),sub.end());
                stk.pop();// 去除'['

                // 获取倍数数字
                string numStr="";
                while( !stk.empty() && isdigit(stk.top())){
                    numStr += stk.top();
                    stk.pop();
                }
                // 反转数字
                reverse(numStr.begin(),numStr.end());
                int cnt = stoi(numStr);

                // 把字符串重复倍数次，再push回去
                while( cnt>0 ){
                    for(char ch:sub){
                        stk.push(ch);
                    }
                    cnt--;
                }
            }
        }
        // 取出栈中的str，输出
        string res="";
        while( !stk.empty() ){
            res+=stk.top();
            stk.pop();
        }
        // 反转字符串
        reverse(res.begin(),res.end());

        return res;
    }
};
```

-

739.每日温度

- 一个整数数组 `temperatures` ，表示每天的温度，返回一个数组 `answer` ，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。

```
输入： temperatures = [73,74,75,71,69,72,76,73]
输出： [1,1,4,2,1,1,0,0]
```

- 思路：

  我们维护一个栈，栈中存储的是**日期的下标 (index)**，而不是温度值本身。
  这个栈有一个关键特性：**栈底到栈顶对应的温度是单调递减的。**

  - 这意味着，栈里存的都是“还没找到下一个更高温度”的日子。
  - 一旦遇到一个比栈顶日子温度更高的当前日子，就说明栈顶那个日子“等到了”升温，可以出栈并计算结果了。

```cpp
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& temperatures) {
        int n = temperatures.size();
        vector<int> ans(n);
        stack<int> stk; // 用于存储温度的下标，需要计算天数差(j-i)
        // 遍历每一天的温度
        for( int i=0; i<n; i++ ){
            // 对于栈顶的那一天(stk.top())，找到了下一个更高温度(当前的第 i 天)
            while( !stk.empty() && temperatures[i] > temperatures[stk.top()]
){
                int previousIndex = stk.top();
                ans[previousIndex]=i - previousIndex; // 记录天数差
                stk.pop();
            }
            stk.push(i); // 当前第i天对应的温度一直递减，则压入栈中
        }
        return ans;
    }
};
```

-